

Q1)

1. Design

```
1  module ALSU(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
2  parameter INPUT_PRIORITY = "A";
3  parameter FULL_ADDER = "ON";
4  input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
5  input [2:0] opcode;
6  input signed [2:0] A, B;
7  output reg [15:0] leds;
8  output reg signed [5:0] out;
9
10 reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
11 reg cin_reg; //Fixed Bug: CIN supposed to be 1 bit unsigned not 2 bits signed
12 reg [2:0] opcode_reg;
13 reg signed [2:0] A_reg, B_reg;
14
15 wire invalid_red_op, invalid_opcode, invalid;
16
17 //Invalid handling
18 assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
19 assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
20 assign invalid = invalid_red_op | invalid_opcode;
21
22 //Registering input signals
23 always @(posedge clk or posedge rst) begin
24     if(rst) begin
25         cin_reg <= 0;
26         red_op_B_reg <= 0;
27         red_op_A_reg <= 0;
28         bypass_B_reg <= 0;
29         bypass_A_reg <= 0;
30         direction_reg <= 0;
31         serial_in_reg <= 0;
32         opcode_reg <= 0;
33         A_reg <= 0;
34         B_reg <= 0;
35     end else begin
36         cin_reg <= cin;
37         red_op_B_reg <= red_op_B;
38         red_op_A_reg <= red_op_A;
39         bypass_B_reg <= bypass_B;
40         bypass_A_reg <= bypass_A;
41         direction_reg <= direction;
42         serial_in_reg <= serial_in;
43         opcode_reg <= opcode;
44         A_reg <= A;
45         B_reg <= B;
46     end
47 end
48
49 //leds output blinking
50 always @(posedge clk or posedge rst) begin
51     if(rst) begin
52         leds <= 0;
53     end else begin
54         if (invalid)
55             leds <= ~leds;
56         else
57             leds <= 0;
58     end
59 end
60
```

```

60
61 //ALSU output processing
62 always @(posedge clk or posedge rst) begin
63     if(rst) begin
64         out <= 0;
65     end
66     else begin
67         if (bypass_A_reg && bypass_B_reg)
68             out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
69         else if (bypass_A_reg)
70             out <= A_reg;
71         else if (bypass_B_reg)
72             out <= B_reg;
73         else if (invalid)
74             out <= 0;
75         else begin
76             case (opcode_reg) //Fixed Bug: changed incorrect checking opcode to opcode_reg
77                 3'h0: begin
78                     if (red_op_A_reg && red_op_B_reg)
79                         out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
80                     else if (red_op_A_reg)
81                         out <= |A_reg;
82                     else if (red_op_B_reg)
83                         out <= |B_reg;
84                     else
85                         out <= A_reg | B_reg;
86                 end
87                 3'h1: begin
88                     if (red_op_A_reg && red_op_B_reg)
89                         out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
90                     else if (red_op_A_reg)
91                         out <= ^A_reg;
92                     else if (red_op_B_reg)
93                         out <= ^B_reg;
94                     else
95                         out <= A_reg ^ B_reg;
96                 end
97                 3'h2: out <= (FULL_ADDER == "ON")? A_reg + B_reg + cin_reg : A_reg + B_reg ; //Fixed Bug: added FULL_ADDER checker
98                 3'h3: out <= A_reg * B_reg;
99                 3'h4: begin
100                     if (direction_reg)
101                         out <= {out[4:0], serial_in_reg};
102                     else
103                         out <= {serial_in_reg, out[5:1]};
104                 end
105                 3'h5: begin
106                     if (direction_reg)
107                         out <= {out[4:0], out[5]};
108                     else
109                         out <= {out[0], out[5:1]};
110                 end
111                 default: out <= 0;
112             endcase
113         end
114     end
115 end
116
117 endmodule

```

2. Do file:

```

1 vlib work
2 vlog ALSU_pkg.sv D_FF.v ALSU.v ALSU_Golden_2.v ALSU_tb.sv +cover -covercells
3 vsim -voptargs=+acc work.ALSU_tb -cover
4 do wave.do
5 coverage save ALSU_tb.ucdb -onexit
6 run -all
7 coverage exclude -src ALSU.v -line 111 -code s
8 coverage exclude -src ALSU.v -line 111 -code b
9 #quit -sim
10 #vcover report ALSU_tb.ucdb -details -annotate -all -output coverage_rpt.txt

```

3. Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functional Checking
ALSU_1	When the reset is asserted, the outputs should Directed at the start of the be low	Directed at the start of the sim, then randomized with constraint that drive the reset to be off most of the simulation time.		A checker in the testbench to make sure the output is correct
ALSU_2	Incase of invalid cases do not occur, when opcode is add, then out should perform the addition on ports A and B taking cin if parameter FULL_ADDER is high	Randomization under constraints on the A and B to have the maximum, minimum and zero values most of the time	Covers: A_CP, B_CP , ALU_cp, A_B_ALU_cross, CIN_ADD_cross, DIRECTION_ALU_cross, SERIAL_IN_SHIFT_cross, RED_OP_A_OR_XOR_cross, RED_OP_B_OR_XOR_cross	Output Checked against golden model
ALSU_3	Incase of bypass_A and bypass_B, if INPUT_PRIORITY is set to A, the output should have value of A	Randomization under constraints on the bypass_A and bypass_B to off most of the time		Output Checked against golden model
ALSU_4	Incase of red_op_A and red_op_B are set , if INPUT_PRIORITY is set to A, the output should have value of Reduction operation on A	Directed testing	ALU_cp, RED_OP_A_OR_XOR_cross, RED_OP_B_OR_XOR_cross	A checker in the testbench to make sure the output is correct
ALSU_5	Incase of invalid cases do occur, the leds should toggle and out is set to zero unless bypass signals are set	Directed testing	ALU_cp, RED_INVLAID_cross	A checker in the testbench to make sure the output is correct

4. Test Bench Code:

Package(added Cross Groups):

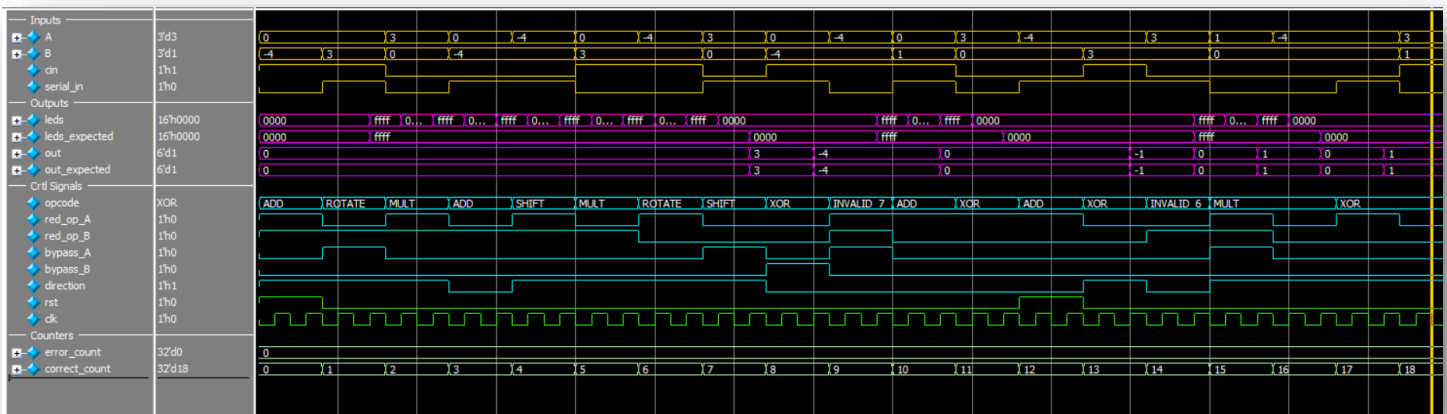
```
1
2 //Added coverpoint for C_in
3 cin_CP: coverpoint cin
4 { option.weight = 0;}
5
6 //Added coverpoint for direction
7 direction_CP: coverpoint direction{
8     option.weight = 0;
9 }
10
11 //Added coverpoint for serial_in
12 serial_in_CP: coverpoint serial_in{
13     option.weight = 0;
14 }
15
16 //Added coverpoint for red_op_A
17 red_op_A_cp: coverpoint red_op_A{
18     option.weight = 0;
19 }
20
21 //Added coverpoint for red_op_B
22 red_op_B_cp: coverpoint red_op_B{
23     option.weight = 0;
24 }
25
26
27 ALU_cp: coverpoint opcode {
28     bins Bins_shift[] = {SHIFT, ROTATE};
29     bins Bins_arith[] = {ADD, MULT};
30     bins Bins_bitwise[] = {OR, XOR};
31     bins Bins_trans = (0 => 1 => 2 => 3 => 4 => 5);
32     illegal_bins Bins_invalid = {INVALID_6, INVALID_7};
33 }
34
35
36
37 //1.Cross coverage for ADD and MULT with A/B permutations
38 A_B_ALU_cross: cross A_CP, B_CP, ALU_cp {
39
40     bins add_mult_perms = binsof(ALU_cp.Bins_arith) &&
41         binsof(A_CP) intersect {MAXPOS, MAXNEG, 0} &&
42         binsof(B_CP) intersect {MAXPOS, MAXNEG, 0};
43     option.cross_auto_bin_max = 0;
44 }
45
46 //2.ALU is addition, c_in should have taken 0 or 1
47 C_ADD_cross: cross ALU_cp, cin_CP{
48
49     bins cin_add = binsof(ALU_cp.Bins_arith) intersect {ADD} &&
50         binsof(cin_CP) intersect {0, 1};
51     option.cross_auto_bin_max = 0;
52 }
53
54
55 //3.ALSU is shifting or rotating, then direction must take 0 or 1
56 DIRECTION_ALU_cross: cross direction_CP, ALU_cp {
57
58     bins direction_ALU = binsof(ALU_cp.Bins_shift) &&
59         binsof(direction_CP) intersect {0, 1};
60     option.cross_auto_bin_max = 0;
61 }
62
63 //4.ALU is shifting, then shift is must take 0 or 1
```

```

62
63 //4.ALSU is shifting, then shift_in must take 0 or 1
64 SERIAL_IN_SHIFT_cross: cross serial_in_CP, ALU_cp {
65     bins serial_in_ALU = binsof(ALU_cp.Bins_shift) intersect {SHIFT} &&
66         binsof(serial_in_CP) intersect {0, 1};
67     option.cross_auto_bin_max = 0;
68 }
69
70 //5.ALSU is OR or XOR and red_op_A is asserted, then A took all walking one patterns
71 //(001, 010, and 100) while B is taking the value 0
72 RED_OP_A_OR_XOR_cross: cross ALU_cp, A_CP, B_CP {
73     bins red_op_A_or_xor = binsof(ALU_cp.Bins_bitwise) &&
74         binsof(A_CP.A_data_walkingones) &&
75         binsof(B_CP.B_data_0);
76     option.cross_auto_bin_max = 0;
77 }
78
79 //6.ALSU is OR or XOR and red_op_B is asserted, then B took all walking one patterns
80 //(001, 010, and 100) while A is taking the value 0
81 RED_OP_B_OR_XOR_cross: cross ALU_cp, A_CP, B_CP {
82     bins red_op_A_or_xor = binsof(ALU_cp.Bins_bitwise) &&
83         binsof(B_CP.B_data_walkingones) &&
84         binsof(A_CP.A_data_0);
85     option.cross_auto_bin_max = 0;
86 }
87
88 //7.invalid case: reduction operation is activated while the opcode is not OR or XOR
89 RED_INVALID_cross: cross ALU_cp, red_op_A_cp, red_op_B_cp {
90     ignore_bins OR_XOR = binsof(red_op_A_cp) &&
91         binsof(red_op_B_cp) &&
92         binsof(ALU_cp.Bins_bitwise);
93
94     ignore_bins trans = binsof(ALU_cp.Bins_trans);
95 }
96
97 endgroup

```

5. QuestaSim



6. Functional Coverage:

```

% hit: 85.71% 100 -
Covergroup instance \\\ALSU_pkg::ALSU_inputs::CovCode
covered/total bins: 95.23% 100 - Uncovered
missing/total bins: 14 15 -
% Hit: 1 15 -
Coverpoint A_CP 93.33% 100 -
covered/total bins: 100.00% 100 - Covered
missing/total bins: 4 4 -
% Hit: 0 4 -
bin A_data_0 100.00% 100 - Covered
bin A_data_max 126 1 - Covered
bin A_data_walkingones[1] 152 1 - Covered
bin A_data_walkingones[2] 4 1 - Covered
default bin A_data_default 2 1 - Covered
Coverpoint B_CP 132 - Occurred
covered/total bins: 100.00% 100 - Covered
missing/total bins: 4 4 -
% Hit: 0 4 -
bin B_data_0 100.00% 100 - Covered
bin B_data_max 118 1 - Covered
bin B_data_walkingones[1] 124 1 - Covered
bin B_data_walkingones[2] 2 1 - Covered
default bin B_data_default 4 1 - Covered
Coverpoint ALU_cp 134 - Occurred
covered/total bins: 85.71% 100 - Uncovered
missing/total bins: 6 7 -
% Hit: 1 7 -
illegal_bin Bins_invalid 85.71% 100 - Occurred
bin Bins_shift[SHIFT] 30 -
bin Bins_shift[ROTATE] 76 1 - Covered
bin Bins_arith[ADD] 74 1 - Covered
bin Bins_arith[MULT] 90 1 - Covered
bin Bins_bitwise[OR] 78 1 - Covered
bin Bins_bitwise[XOR] 6 1 - Covered
bin Bins_trans 70 1 - Covered
0 1 - ZERO

TOTAL COVERGROUP COVERAGE: 95.23% COVERGROUP TYPES: 1

Total Coverage By Instance (filtered view): 95.23%

```

Note: Couldn't have Bins_trans to be covered even with directed testing

Cross Coverage: (ALU_CP not 100% due to trans_bins not working)

INST \ALSU_pkg::ALSU_inputs::CovCode	98.57%	100	98.57%		✓
+	CVP A_CP	100.00%	100	100.00...	✓
+	CVP B_CP	100.00%	100	100.00...	✓
+	CVP cin_CP	100.00%	100	100.00...	✓
+	CVP direction_CP	100.00%	100	100.00...	✓
+	CVP serial_in_CP	100.00%	100	100.00...	✓
+	CVP red_op_A_cp	100.00%	100	100.00...	✓
+	CVP red_op_B_cp	100.00%	100	100.00...	✓
+	CVP ALU_cp	85.71%	100	85.71%	✓
+	CROSS A_B_ALU_cross	100.00%	100	100.00...	✓
+	CROSS C_ADD_cross	100.00%	100	100.00...	✓
+	CROSS DIRECTION_ALU_cross	100.00%	100	100.00...	✓
+	CROSS SERIAL_IN_SHIFT_cross	100.00%	100	100.00...	✓
+	CROSS RED_OP_A_OR_XOR_cross	100.00%	100	100.00...	✓
+	CROSS RED_OP_B_OR_XOR_cross	100.00%	100	100.00...	✓
+	CROSS RED_INVLAID_cross	100.00%	100	100.00...	✓

Q2)

```

1  module assertions();
2
3      //1
4      property p_a_rise_b;
5          @(posedge clk) $rose(a) |-> ##2 (b);
6      endproperty
7
8      //2
9      property p_a_rise_b;
10         @(posedge clk) (a && b) |-> ##[1:3] c;
11     endproperty
12
13     //3
14     sequence p_s11b;
15         @(posedge clk) 1 |-> ##2 (!b);
16     endsequence
17
18     //4
19     property p_3_8_decoder_Y;
20         @(posedge clk) $onehot(Y);
21     endproperty
22
23     property p_4_to_2_priority_encode_valid;
24         @(posedge clk) (~D) |>= (~valid);
25     endproperty
26
27 endmodule

```

Q3)

1. Top module:



```
1  module counter_top();
2  // 1. Generate the clock
3      bit clk;
4      initial begin
5          clk = 0;
6          forever
7              #1 clk = ~clk;
8      end
9  // 2. instantiate the interface, and pass the clock
10     counter_if c_if(clk);
11 // 3. instantiate the tb, DUT, monitor, and pass the interface
12     counter_tb tb(c_if);
13     counter DUT(c_if);
14     counter_sva count_sva(c_if);
15
16 // 4. bind the SVA module to the design, and pass the interface
17     bind counter counter_sva counter_sva_inst(c_if);
18
19 endmodule
```


2. Interface

```
1 import counter_pkg::*;
2
3 interface counter_if(input clk);
4
5     // the internal signals of the interface
6     logic rst_n;                //(active low sync rst)
7     logic load_n;               //(active low load)
8     logic up_down;              //high then increment counter, else decrement
9     logic ce;                   //(clock enable signal)
10    logic [WIDTH-1:0] data_load; //(load data to count_out output when the load_n signal is asserted)
11    logic [WIDTH-1:0] count_out;
12    logic max_count;             //counter reaches the maximum value, this signal is high, else low)
13    logic zero;                  //counter reaches the minimum value, this signal is high, else low)
14
15    //Modports
16    modport DUT (input clk, rst_n, load_n, up_down, ce, data_load,
17                output count_out, max_count, zero);
18
19    modport TEST (input clk, count_out, max_count, zero,
20                 output rst_n, load_n, up_down, ce, data_load);
21
22    modport MONITOR (input clk, count_out, max_count, zero,
23                    rst_n, load_n, up_down, ce, data_load);
24
25
26
27
28 endinterface
```

3. Design

```
8 module counter (counter_if.DUT c_if);
9     parameter WIDTH = 4;           //(Valid values: 4, 6, 8, default: 4)
10
11     always @(posedge c_if.clk or negedge c_if.rst_n) begin
12         if (~c_if.rst_n)            //Modification: changed comparison "!" to bitwise NOT "~"
13             c_if.count_out <= 0;
14         else if (~c_if.load_n)       //Modification: changed comparison "!" to bitwise NOT "~"
15             c_if.count_out <= c_if.data_load;
16         else if (c_if.ce)begin       //Fixed BUG: added Begin and End statements
17             if (c_if.up_down)
18                 c_if.count_out <= c_if.count_out + 1;
19             else
20                 c_if.count_out <= c_if.count_out - 1;
21         end
22     end
23
24     assign c_if.max_count = (c_if.count_out == {WIDTH{1'b1}})? 1:0;
25     assign c_if.zero = (c_if.count_out == 0)? 1:0;
26
27 endmodule
```

4. Assertions:

```
1  import counter_pkg::*;
2  module counter_sva(counter_if.DUT c_if);
3
4      //1.When the load control signal is active, then the dout has the value of the din
5      property p_load_active_count_out;
6          @(posedge c_if.clk) disable iff(~c_if.rst_n) (~c_if.load_n) |>= (c_if.count_out == $past(c_if.data_load));
7      endproperty
8
9      //2.When the load control signal is not active, and the enable is off then the dout does not change
10     property p_load_inactive_enable_off;
11         @(posedge c_if.clk) disable iff(~c_if.rst_n) ((c_if.load_n) && (~c_if.ce)) |>= $stable(c_if.count_out);
12     endproperty
13
14     //3.When the load control signal is not active and the enable is active, and the up_down is high then the dout is incremented.
15     property p_load_inactive_enable_on_updown_1;
16         @(posedge c_if.clk) disable iff(~c_if.rst_n || $isunknown(c_if.count_out))
17         (c_if.load_n && c_if.ce && c_if.up_down) |>= (c_if.count_out == $past(c_if.count_out) + 1'b1);
18     endproperty
19
20     //4.When the load control signal is not active and the enable is active, and the up_down is low
21     // then the dout is decremented.
22     property p_load_inactive_enable_on_updown_0;
23         @(posedge c_if.clk) disable iff(~c_if.rst_n) (c_if.load_n && c_if.ce && ~(c_if.up_down)) |>= (c_if.count_out == $past(c_if.count_out) - 1'b1);
24     endproperty
25
26     always_comb begin
27         //5.When the asynchronous reset is asserted then the counter output is tied to low at the same instant.
28         if(~c_if.rst_n) begin
29             assert final (c_if.count_out == 0);
30             cover final (c_if.count_out == 0);
31         end
32
33         //6.max_count output is high when the counter output is maximum.
34         if(c_if.count_out == {WIDTH{1'b1}})begin
35             assert final (c_if.max_count);
36             cover final (c_if.max_count);
37         end
38
39         //7.zero output is high when the counter output is zero.
40         if(c_if.count_out == 4'b0000)begin
41             assert final (c_if.zero);
42             cover final (c_if.zero);
43         end
44     end
45
46     load_active_count_out_assertion : assert property (p_load_active_count_out);
47     load_inactive_enable_off_assertion : assert property (p_load_inactive_enable_off);
48     load_inactive_enable_on_updown_1_assertion : assert property (p_load_inactive_enable_on_updown_1);
49     load_inactive_enable_on_updown_0_assertion : assert property (p_load_inactive_enable_on_updown_0);
50
51     load_active_count_out_coverage : cover property (p_load_active_count_out);
52     load_inactive_enable_off_coverage : cover property (p_load_inactive_enable_off);
53     load_inactive_enable_on_updown_1_coverage : cover property (p_load_inactive_enable_on_updown_1);
54     load_inactive_enable_on_updown_0_coverage : cover property (p_load_inactive_enable_on_updown_0);
55
56
57
58 endmodule
```

5. Test Bench

```
1  import counter_pkg::*;
2
3  module counter_tb(counter_if,TEST c_if);
4
5
6      Counter_inputs inputs = new();
7
8      initial begin
9          assert(inputs.randomize());
10         //COUNTER_1
11         c_if.rst_n = 0;
12         c_if.load_n = inputs.load_n;
13         c_if.up_down = inputs.up_down;
14         c_if.ce = inputs.ce;
15         c_if.data_load = inputs.data_load;
16
17         @(negedge c_if.clk);
18         inputs.count_out = c_if.count_out;
19         inputs.CovCode.sample();
20
21         //COUNTER_2, 3, 4, 5
22         for(int i = 0; i < 200; i++)begin
23             assert(inputs.randomize());
24             c_if.rst_n = inputs.rst_n;
25             c_if.load_n = inputs.load_n;
26             c_if.up_down = 1; inputs.up_down = 1;
27             c_if.ce = inputs.ce;
28             c_if.data_load = inputs.data_load;
29
30             @(negedge c_if.clk);
31             inputs.count_out = c_if.count_out;
32             inputs.CovCode.sample();
33         end
34
35         for(int i = 0; i < 200; i++)begin
36             assert(inputs.randomize());
37             c_if.rst_n = inputs.rst_n;
38             c_if.load_n = inputs.load_n;
39             c_if.up_down = 0; inputs.up_down = 0;
40             c_if.ce = inputs.ce;
41             c_if.data_load = inputs.data_load;
42
43             @(negedge c_if.clk);
44             inputs.count_out = c_if.count_out;
45             inputs.CovCode.sample();
46         end
47
48         //COUNTER_6
49         c_if.rst_n = 1; c_if.load_n = 0; c_if.ce = 1; c_if.data_load = 4'b1111;
50         inputs.rst_n = 1; inputs.load_n = 0; inputs.ce = 1; inputs.data_load = 4'b1111;
51         @(negedge c_if.clk);
52         inputs.count_out = c_if.count_out;
53         inputs.CovCode.sample();
54
55         //COUNTER_7
56         c_if.rst_n = 1; c_if.load_n = 0; c_if.ce = 1; c_if.data_load = 0;
57         inputs.rst_n = 1; inputs.load_n = 0; inputs.ce = 1; inputs.data_load = 0;
58         @(negedge c_if.clk);
59         inputs.count_out = c_if.count_out;
60         inputs.CovCode.sample();
61
62         //Additional Tests for 100% Coverage
63         assert(inputs.randomize());
64         c_if.rst_n = 0; inputs.rst_n = 0;
65         c_if.load_n = inputs.load_n;
66         c_if.up_down = inputs.up_down;
67         c_if.ce = inputs.ce;
68         c_if.data_load = inputs.data_load;
69         @(negedge c_if.clk);
70         inputs.count_out = c_if.count_out;
71         inputs.CovCode.sample();
72
73
74         assert(inputs.randomize());
75         c_if.rst_n = inputs.rst_n;
76         c_if.load_n = inputs.load_n;
77         c_if.up_down = 1; inputs.up_down = 1;
78         c_if.ce = inputs.ce;
79         c_if.data_load = inputs.data_load;
80         @(negedge c_if.clk);
81         inputs.count_out = c_if.count_out;
82         inputs.CovCode.sample();
83         $stop;
84
85     end
86
87 endmodule
```

6. Package

```
1  package counter_pkg;
2
3  parameter WIDTH = 4;
4  localparam MAX_VAL = (1 << WIDTH) - 1;
5
6  class Counter_inputs;
7
8      rand logic rst_n;                //(active low sync rst)
9      rand logic load_n;               //(active low load)
10     rand logic up_down;               //high then increment counter, else decrement
11     rand logic ce;                   //(clock enable signal
12     rand logic [WIDTH-1:0] data_load; //(load data to count_out output when the load_n signal is asserted)
13
14     logic [WIDTH-1:0] count_out;
15
16     constraint c_rst_n{
17         rst_n dist{0:/20, 1:/80};
18     }
19
20     constraint c_load_n{
21         load_n dist{0:/30, 1:/70};
22     }
23
24     constraint c_ce{
25         ce dist{0:/30, 1:/70};
26     }
27
28     covergroup CovCode;
29
30         load_data_cp: coverpoint data_load iff(rst_n && ~load_n);
31
32         count_out_updown_high_cp : coverpoint count_out iff(rst_n && ce && up_down);
33
34         count_out_trans_updown_high_cp : coverpoint count_out iff(rst_n && ce && up_down){
35             bins OV_MAX_ZERO = (4'hf => 0);
36         }
37
38         count_out_updown_low_cp : coverpoint count_out iff(rst_n && ce && ~up_down);
39
40         count_out_trans_updown_low_cp : coverpoint count_out iff(rst_n && ce && ~up_down){
41             bins OV_ZERO_MAX = (0 => 4'hf);
42         }
43
44     endgroup
45
46
47     function new(logic rst_n = 1, logic load_n = 1, logic up_down = 1, logic ce = 0,
48         logic [WIDTH-1:0] data_load = 0);
49
50         this.rst_n = rst_n;
51         this.load_n = load_n;
52         this.up_down = up_down;
53         this.ce = ce;
54         this.data_load = data_load;
55
56         CovCode = new;
57
58     endfunction
59
60
61 endclass
62
63 endpackage
```

7. Assertion Coverage

ame	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph
▲ /counter_top/DUT/counter_sva_inst/#ublk#179177105#28/i_cover__30	SVA	✓	Off	80	1	Unli...	1	100%	
▲ /counter_top/DUT/counter_sva_inst/#ublk#179177105#34/i_cover__36	SVA	✓	Off	34	1	Unli...	1	100%	
▲ /counter_top/DUT/counter_sva_inst/#ublk#179177105#40/i_cover__42	SVA	✓	Off	172	1	Unli...	1	100%	
▲ /counter_top/DUT/counter_sva_inst/load_active_count_out_coverage	SVA	✓	Off	73	1	Unli...	1	100%	
▲ /counter_top/DUT/counter_sva_inst/load_inactive_enable_off_coverage	SVA	✓	Off	55	1	Unli...	1	100%	
▲ /counter_top/DUT/counter_sva_inst/load_inactive_enable_on_updown_1_coverage	SVA	✓	Off	46	1	Unli...	1	100%	
▲ /counter_top/DUT/counter_sva_inst/load_inactive_enable_on_updown_0_coverage	SVA	✓	Off	53	1	Unli...	1	100%	
▲ /counter_top/count_sva/#ublk#179177105#28/i_cover__30	SVA	✓	Off	80	1	Unli...	1	100%	
▲ /counter_top/count_sva/#ublk#179177105#34/i_cover__36	SVA	✓	Off	34	1	Unli...	1	100%	
▲ /counter_top/count_sva/#ublk#179177105#40/i_cover__42	SVA	✓	Off	172	1	Unli...	1	100%	
▲ /counter_top/count_sva/load_active_count_out_coverage	SVA	✓	Off	73	1	Unli...	1	100%	
▲ /counter_top/count_sva/load_inactive_enable_off_coverage	SVA	✓	Off	55	1	Unli...	1	100%	
▲ /counter_top/count_sva/load_inactive_enable_on_updown_1_coverage	SVA	✓	Off	46	1	Unli...	1	100%	
▲ /counter_top/count_sva/load_inactive_enable_on_updown_0_coverage	SVA	✓	Off	53	1	Unli...	1	100%	

8. Assertion tests

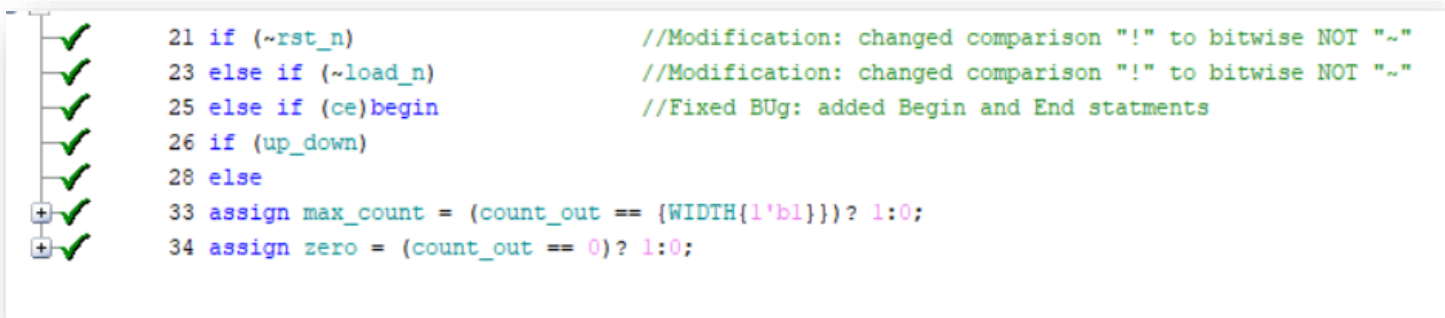
Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active
▲ /counter_top/tb/#ublk#95084642#8/immed__9	Immediate	SVA	on	0	1	
▲ /counter_top/tb/#ublk#95084642#8/immed__64	Immediate	SVA	on	0	1	
▲ /counter_top/tb/#ublk#95084642#8/immed__75	Immediate	SVA	on	0	1	
▲ /counter_top/tb/#ublk#95084642#8/#anonblk#95084642#22#4#/#ublk#95084642#22/imm...	Immediate	SVA	on	0	1	
▲ /counter_top/tb/#ublk#95084642#8/#anonblk#95084642#35#4#/#ublk#95084642#35/imm...	Immediate	SVA	on	0	1	
+▲ /counter_top/DUT/counter_sva_inst/load_active_count_out_assertion	Concurrent	SVA	on	0	1	
+▲ /counter_top/DUT/counter_sva_inst/load_inactive_enable_off_assertion	Concurrent	SVA	on	0	1	
+▲ /counter_top/DUT/counter_sva_inst/load_inactive_enable_on_updown_1_assertion	Concurrent	SVA	on	0	1	
+▲ /counter_top/DUT/counter_sva_inst/load_inactive_enable_on_updown_0_assertion	Concurrent	SVA	on	0	1	
▲ /counter_top/DUT/counter_sva_inst/#ublk#179177105#28/immed__29	Immediate	SVA	on	0	1	
▲ /counter_top/DUT/counter_sva_inst/#ublk#179177105#34/immed__35	Immediate	SVA	on	0	1	
▲ /counter_top/DUT/counter_sva_inst/#ublk#179177105#40/immed__41	Immediate	SVA	on	0	1	
+▲ /counter_top/count_sva/load_active_count_out_assertion	Concurrent	SVA	on	0	1	
+▲ /counter_top/count_sva/load_inactive_enable_off_assertion	Concurrent	SVA	on	0	1	
+▲ /counter_top/count_sva/load_inactive_enable_on_updown_1_assertion	Concurrent	SVA	on	0	1	
+▲ /counter_top/count_sva/load_inactive_enable_on_updown_0_assertion	Concurrent	SVA	on	0	1	
▲ /counter_top/count_sva/#ublk#179177105#28/immed__29	Immediate	SVA	on	0	1	
▲ /counter_top/count_sva/#ublk#179177105#34/immed__35	Immediate	SVA	on	0	1	
▲ /counter_top/count_sva/#ublk#179177105#40/immed__41	Immediate	SVA	on	0	1	

9. Functional Coverage

[-] /counter_pkg/Counter_inputs	100.00%					
[-] TYPE CovCode	100.00%	100	100.00...		✓	auto(0)
CVP CovCode::load_data_cp	100.00%	100	100.00...		✓	
CVP CovCode::count_out_updown_high_cp	100.00%	100	100.00...		✓	
CVP CovCode::count_out_trans_updown_hig...	100.00%	100	100.00...		✓	
CVP CovCode::count_out_updown_low_cp	100.00%	100	100.00...		✓	
CVP CovCode::count_out_trans_updown_low...	100.00%	100	100.00...		✓	
[-] INST \counter_pkg::Counter_inputs::CovCo...	100.00%	100	100.00...		✓	
+ CVP load_data_cp	100.00%	100	100.00...		✓	
+ CVP count_out_updown_high_cp	100.00%	100	100.00...		✓	
+ CVP count_out_trans_updown_high_cp	100.00%	100	100.00...		✓	
+ CVP count_out_updown_low_cp	100.00%	100	100.00...		✓	
+ CVP count_out_trans_updown_low_cp	100.00%	100	100.00...		✓	

10. Code Coverage

Branch Coverage:

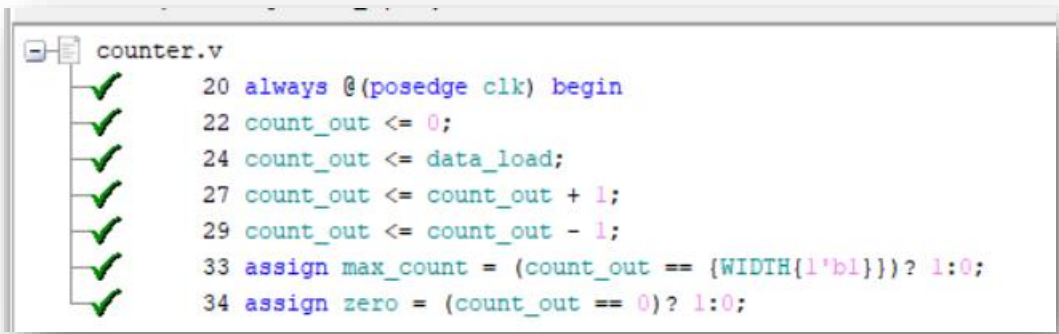


A screenshot of a code editor showing the file `counter.v`. The code is as follows:

```
21 if (~rst_n) //Modification: changed comparison "!" to bitwise NOT "~"
23 else if (~load_n) //Modification: changed comparison "!" to bitwise NOT "~"
25 else if (ce)begin //Fixed BUG: added Begin and End statements
26 if (up_down)
28 else
33 assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
34 assign zero = (count_out == 0)? 1:0;
```

On the left side of the code, there is a vertical list of green checkmarks indicating 100% branch coverage for each line of code.

Statement Coverage:

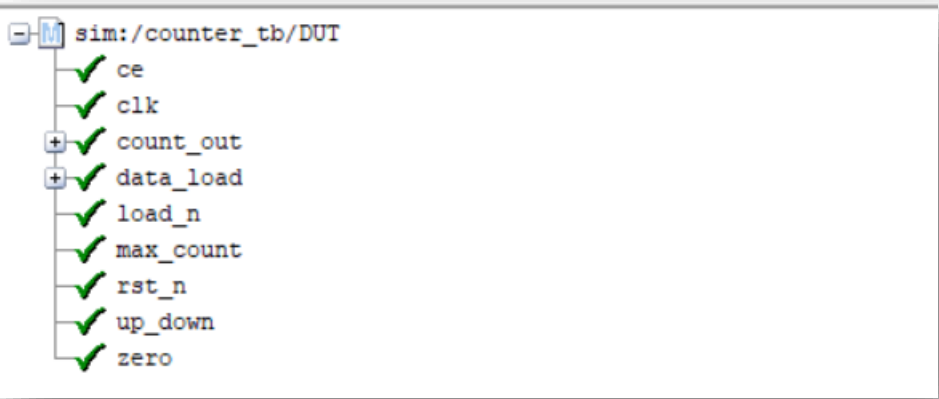


A screenshot of a code editor showing the file `counter.v`. The code is as follows:

```
20 always @(posedge clk) begin
22 count_out <= 0;
24 count_out <= data_load;
27 count_out <= count_out + 1;
29 count_out <= count_out - 1;
33 assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
34 assign zero = (count_out == 0)? 1:0;
```

On the left side of the code, there is a vertical list of green checkmarks indicating 100% statement coverage for each line of code.

Toggle Coverage:

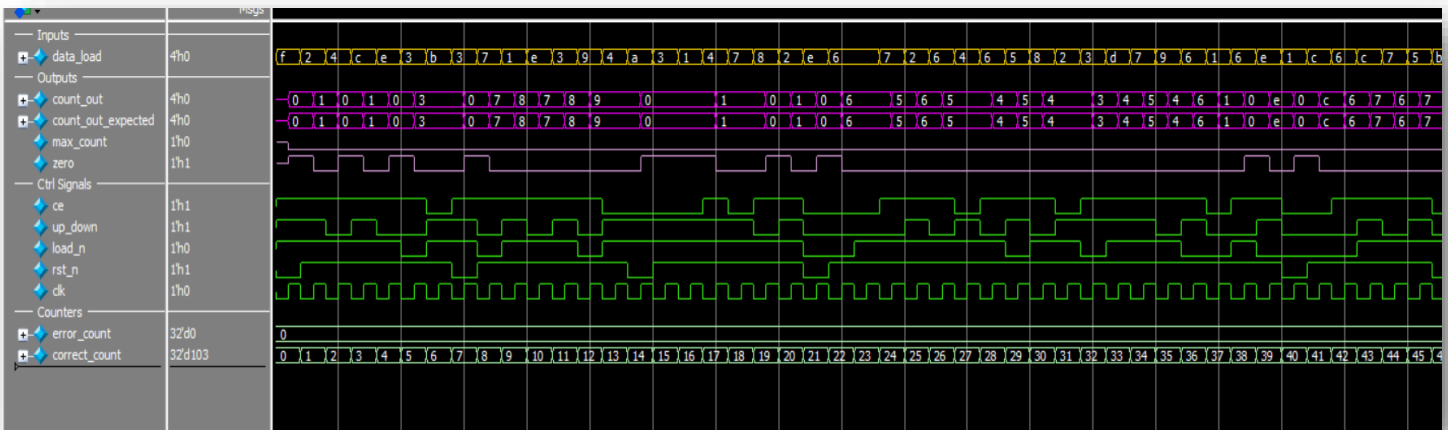


A screenshot of a code editor showing the file `sim:/counter_tb/DUT`. The code is as follows:

```
ce
clk
count_out
data_load
load_n
max_count
rst_n
up_down
zero
```

On the left side of the code, there is a vertical list of green checkmarks indicating 100% toggle coverage for each signal.

11. Questasim Waveform



Note: this is the waveform of half the test. I settled with this half for readability

12. Coverage Report:

Branch Coverage:

```
=====
=== Instance: /\counter_tb#DUT
=== Design Unit: work.counter
=====
Branch Coverage:
  Enabled Coverage      Bins      Hits      Misses      Coverage
  -----
  Branches              10        10         0      100.00%

=====Branch Details=====
Branch Coverage for instance /\counter_tb#DUT

  Line      Item      Count      Source
  ----      -
  File counter.v
  -----IF Branch-----
  21              103      Count coming in to IF
  21              13      if (~rst_n)           //Modification: changed comparison "!" to bitwise NOT "~"
  23              31      else if (~load_n)      //Modification: changed comparison "!" to bitwise NOT "~"
  25              43      else if (ce)begin      //Fixed BUG: added Begin and End statments
  25              16      All False Count
Branch totals: 4 hits of 4 branches = 100.00%

  -----IF Branch-----
  26              43      Count coming in to IF
  26              17      if (up_down)
  28              26      else
Branch totals: 2 hits of 2 branches = 100.00%
```

Statement Coverage:

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	7	7	0	100.00%

=====Statement Details=====

Statement Coverage for instance /\counter_tb#DUT --

Toggle & Total Coverage:

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	30	30	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\counter_tb#DUT --

Node	1H->0L	0L->1H	"Coverage"
ce	1	1	100.00
clk	1	1	100.00
count_out[3-0]	1	1	100.00
data_load[0-3]	1	1	100.00
load_n	1	1	100.00
max_count	1	1	100.00
rst_n	1	1	100.00
up_down	1	1	100.00
zero	1	1	100.00

Total Node Count = 15

Toggled Node Count = 15

Untoggled Node Count = 0

Toggle Coverage = 100.00% (30 of 30 bins)

Total Coverage By Instance (filtered view): 100.00%

Q4)

1. Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functional Checking
CONFIG_REG_1	When the reset is asserted, the values of all the registers should be Reset values	Directed at the start of the sim		A checker in the testbench to make sure the output is correct
CONFIG_REG_2	Set Write to one to check Write data into registers then setting it to zero and checking the written values	randomized through the simulation		A checker in the testbench to make sure the output is correct

2. Package

```
1 package config_reg_pkg;
2
3     typedef enum logic[2:0] { adc0_reg, adc1_reg, temp_sensor0_reg, temp_sensor1_reg,
4                             analog_test, digital_test, amp_gain, digital_config } reset_v;
5
6     parameter WIDTH = 16;
7
8     class reg_inputs;
9
10         logic[WIDTH-1:0] reset_assoc[reset_v];
11
12         rand logic reset;
13         rand logic write;
14         rand logic [15:0] data_in;
15         rand reset_v address;
16
17         constraint reset_c{
18             reset dist {0:/90, 1:/10};
19         }
20
21
22     function new(logic reset = 0, write = 0, data_in = 0);
23         this.reset = reset;
24         this.write = write;
25         this.data_in = data_in;
26
27         this.reset_assoc[adc0_reg] = 16'hFFFF;
28         this.reset_assoc[adc1_reg] = 16'h0;
29         this.reset_assoc[temp_sensor0_reg] = 16'h0;
30         this.reset_assoc[temp_sensor1_reg] = 16'h0;
31         this.reset_assoc[analog_test] = 16'hABCD;
32         this.reset_assoc[digital_test] = 16'h0;
33         this.reset_assoc[amp_gain] = 16'h0;
34         this.reset_assoc[digital_config] = 16'h1;
35
36     endfunction
37
38
39     endclass
40
41 endpackage
```

3. Test bench for the Whole Design

```
1  import config_reg_pkg::*;
2
3  module Config_reg_tb();
4
5
6      //Inputs
7      logic clk;
8      logic reset;
9      logic write;
10     logic [15:0] data_in;
11     reset_v address;
12     //Outputs
13     logic [15:0] data_out;
14
15     //Debug Variables
16     logic [WIDTH-1:0] data_to_write_array[reset_v];
17     logic [WIDTH-1:0] config_reg_expected[reset_v];
18     logic [WIDTH-1:0] read_data_queue[$];
19     logic [WIDTH-1:0] data;
20
21
22     config_reg dut(
23         .clk(clk),
24         .reset(reset),
25         .write(write),
26         .data_in(data_in),
27         .address(address),
28         .data_out(data_out)
29     );
30
31     reg_inputs obj = new();
32     int error_count, correct_count;
33
34     initial begin
35         clk = 0;
36         forever
37             #1 clk = ~clk;
38     end
39
40     initial begin
41         error_count = 0; correct_count = 0;
42
43         //check reset values
44         checkReset();
45
46         //Stimulate random input data
47         stimulus_gen();
48         golden_model();
49
50         for(int i = 0; i < address.num; i++)begin
51             write = 1;
52             data_in = data_to_write_array[address];
53             @(negedge clk)
54             address = address.next;
55         end
56
57         address = address.first;
58
59         for(int i = 0; i < address.num; i++)begin
60             write = 0;
61             @(negedge clk);
62             checkReg(address);
63             read_data_queue.push_front(data_out);
64             address = address.next;
65         end
66
67         end
68
69         for(int i = 0; read_data_queue.size(); i++)begin
70             data = read_data_queue.pop_back();
71             $display("Data Read:%h", i, data);
72         end
73
74         $display("Total Tests:%d | Correct Tests:%d | Error Test:%d", correct_count + error_count, correct_count, error_count);
75         $stop;
76
77     end
78
```

```

78
79     task checkReset();
80
81         write = 0; reset = 1;
82         address = address.first;
83         data_in = 0;
84
85         $display("=====Reset Check=====\\n");
86         for(int i = 0; i < address.num; i++)begin
87             @(negedge clk);
88             if(data_out != obj.reset_assoc[address])begin
89                 error_count++;
90                 $display("Address:%s\\n| Data Out: Expcted:%h | DUT:%h <-----Error", address.name, obj.reset_assoc[address], data_out);
91                 // $stop;
92             end
93             else begin
94                 $display("Address:%s\\n| Data Out: Expcted:%h | DUT:%h", address.name, obj.reset_assoc[address], data_out);
95                 correct_count++;
96             end
97
98             address = address.next;
99         end
100         $display("=====End Reset Check=====\\n");
101         reset = 0;
102     endtask
103
104     task stimulus_gen();
105
106         address = address.first;
107
108         for(int i = 0; i < address.num; i++)begin
109             data_to_write_array[address] = $random;
110             address = address.next;
111         end
112     endtask
113
114     task golden_model();
115         address = address.first;
116
117         for(int i = 0; i < address.num; i++)begin
118             config_reg_expected[address] = data_to_write_array[address];
119             address = address.next;
120         end
121     endtask
122
123     task checkReg(input reset_v address);
124
125         if(data_out != config_reg_expected[address])begin
126             error_count++;
127             $display("Address:%s\\n| Data Out: Expcted:%h | DUT:%h <-----Error", address.name, config_reg_expected[address], data_out);
128             // $stop;
129         end
130         else begin
131             $display("Address:%s\\n| Data Out: Expcted:%h | DUT:%h", address.name, config_reg_expected[address], data_out);
132             correct_count++;
133         end
134
135     endtask
136
137
138 endmodule

```

4. Bugs Found

1. Incorrect Reset Value during 1st reset of Register : **analog_test**

a. Design Input for Bug to Appear:

- i. Reset = 1 & Write = 0 & data_in = 0 & address = 4

b. Expected Behavior:

- i. Data_out should have Value: 0xABCD

c. Observed Behavior:

- i. Data_out has value: 0xABCC

2. Incorrect write assignment in Register: **adc0_reg**

a. Design Input for Bug to Appear:

- i. Reset = 0 & write = 1 & data_in = 0x3524 & address = 0
- ii. Reset = 0 & write = 0 & data_in = 0 & address = 0

b. Expected Behavior:

- i. Data_out should have Value: 0x3524

c. Observed Behavior:

- i. Data_out has value: 0xB524

Note: 15th bit (MSB) is always set to 1.

3. Incorrect write assignment in Register: **adc1_reg**

a. Design Input for Bug to Appear:

- i. Reset = 0 & write = 1 & data_in = 0x5e81 & address = 1
- ii. Reset = 0 & write = 0 & data_in = 0 & address = 1

b. Expected Behavior:

- i. Data_out should have Value: 0x5e81

c. Observed Behavior:

- i. Data_out has value: 0x815e

Note: Hexa's assignments order is not correct (1st switched with 3rd and 2nd switched with 4th).

4. Incorrect write assignment in Register: **temp_sensor0_reg**

a. Design Input for Bug to Appear:

- i. Reset = 0 & write = 1 & data_in = 0xd609 & address = 2
- ii. Reset = 0 & write = 0 & data_in = 0 & address = 2

b. Expected Behavior:

- i. Data_out should have Value: 0xd609

c. Observed Behavior:

- i. Data_out has value: 0xac12

5. Incorrect write assignment in Register: **digital_test**

a. Design Input for Bug to Appear:

- i. Reset = 0 & write = 1 & data_in = 0x998d & address = 5
- ii. Reset = 0 & write = 0 & data_in = 0 & address = 5

b. Expected Behavior:

- i. Data_out should have Value: 0x998d

c. Observed Behavior:

- i. Data_out has value: 0x8465

6. Incorrect write assignment in Register: **amp_gain**

a. Design Input for Bug to Appear:

- i. Reset = 0 & write = 1 & data_in = 0x8465 & address = 6
- ii. Reset = 0 & write = 0 & data_in = 0 & address = 6

b. Expected Behavior:

- i. Data_out should have Value: 0x8465

c. Observed Behavior:

- i. Data_out has value: 0x998d

Note from 6, 5: address addresses of amp_gain & digital_test are switched

7. Incorrect reset value in 2nd reset assignment in Register: **temp_sensor1_reg**

a. Design Input for Bug to Appear:

- i. Reset = 1 & write = 0 & data_in = 0x0 & address = 3
- ii. Reset = 0 & write = 1 & data_in = 0x5663 & address = 3
- iii. Reset = 1 & write = 0 & data_in = 0x0 & address = 3

b. Expected Behavior:

- i. Data_out should have Value: 0x0

c. Observed Behavior:

- i. Data_out has value: 0x5663

Note: It saves the last data written into it.

8. Incorrect write assignment in Register: **digital_config**

d. Design Input for Bug to Appear:

- i. Reset = 0 & write = 1 & data_in = 0x870a & address = 7
- ii. Reset = 0 & write = 0 & data_in = 0 & address = 7

e. Expected Behavior:

- i. Data_out should have Value: 0x870a

f. Observed Behavior:

- i. Data_out has value: 0x170a

Note: 15th bit (MSB) is always set to 0.

9. Directed Test

```
1  import config_reg_pkg::*;
2
3  module Config_reg_directed_tb();
4
5      //Inputs
6      logic clk;
7      logic reset;
8      logic write;
9      logic [15:0] data_in;
10     reset_v address;
11     //Outputs
12     logic [15:0] data_out;
13
14     config_reg dut(
15         .clk(clk),
16         .reset(reset),
17         .write(write),
18         .data_in(data_in),
19         .address(address),
20         .data_out(data_out)
21     );
22
23     initial begin
24         clk = 0;
25         forever
26             #1 clk = ~clk;
27     end
28
29     initial begin
30
31         //1.Incorrect Reset Value during 1st reset of Register : analog_test
32         reset = 1; write = 0; data_in = 0; address = analog_test;
33         @(negedge clk);
34
35         $display("Address:%s\n| Data Out: Expcted:0xABCD | DUT:%h <-----Error", address.name, data_out);
36
37         //2.Incorrect write assignment in Register: adc0_reg
38         reset = 0; write = 1; data_in = 16'h3524 ; address = adc0_reg;
39         @(negedge clk);
40         reset = 0; write = 0; data_in = 16'h0 ; address = adc0_reg;
41         @(negedge clk);
42         $display("Address:%s\n| Data Out: Expcted:0x3524 | DUT:%h <-----Error", address.name, data_out);
43
44         //3.Incorrect write assignment in Register: adc1_reg
45         reset = 0; write = 1; data_in = 16'h5e81 ; address = adc1_reg;
46         @(negedge clk);
47         reset = 0; write = 0; data_in = 16'h0 ; address = adc1_reg;
48         @(negedge clk);
49         $display("Address:%s\n| Data Out: Expcted:0x5e81 | DUT:%h <-----Error", address.name, data_out);
50
51         //4.Incorrect write assignment in Register: temp_sensor0_reg
52         reset = 0; write = 1; data_in = 16'hd609 ; address = temp_sensor0_reg;
53         @(negedge clk);
54         reset = 0; write = 0; data_in = 16'h0 ; address = temp_sensor0_reg;
55         @(negedge clk);
56         $display("Address:%s\n| Data Out: Expcted:0xd609 | DUT:%h <-----Error", address.name, data_out);
57     end
```

```

58
59 //5 & 6.Switched write assignment in Register: amp_gain & digital_test
60 reset = 0; write = 1; data_in = 16'h8465 ; address = amp_gain;
61 @(negedge clk);
62 reset = 0; write = 1; data_in = 16'hd998d ; address = digital_test;
63 @(negedge clk);
64
65 reset = 0; write = 0; data_in = 16'h0 ; address = amp_gain;
66 @(negedge clk);
67 $display("Address:%s\n| Data Out: Expcted:0x8465 | DUT:%h <-----Error", address.name, data_out);
68
69 reset = 0; write = 0; data_in = 16'h0 ; address = digital_test;
70 @(negedge clk);
71 $display("Address:%s\n| Data Out: Expcted:0xd998d | DUT:%h <-----Error", address.name, data_out);
72
73 //7.Incorrect reset value in 2nd reset assignment in Register: temp_sensor1_reg
74 reset = 1; write = 0; data_in = 16'h0 ; address = temp_sensor1_reg;
75 @(negedge clk);
76 reset = 0; write = 1; data_in = 16'h1111 ; address = temp_sensor1_reg;
77 @(negedge clk);
78 reset = 1; write = 0; data_in = 16'h0 ; address = temp_sensor1_reg;
79 @(negedge clk);
80
81 $display("Address:%s\n| Data Out: Expcted:0x0 | DUT:%h <-----Error", address.name, data_out);
82
83 //8.Incorrect write assignment in Register: digital_config
84 reset = 0; write = 1; data_in = 16'h870a ; address = digital_config;
85 @(negedge clk);
86 reset = 0; write = 0; data_in = 16'h0 ; address = digital_config;
87 @(negedge clk);
88 $display("Address:%s\n| Data Out: Expcted:0x870a | DUT:%h <-----Error", address.name, data_out);
89
90 $$display("Total Of 8 Errors");
91 $stop;
92 end
93
94
95 endmodule

```